LibreChat User Manual

LibreChat runs as a set of Docker Compose services and connects to an LLM model served separately by aiDAPTIV middleware. This manual uses our repository, which already includes the aiDAPTIV KV-cache settings integrated to LibreChat.

Part 1: Before You Begin

Setting up Docker Desktop

If you don't already have Docker installed, get Docker Desktop first, everything else in this manual depends on it.

1. Download Docker Desktop from the official site: https://www.docker.com/products/docker-desktop/
2. Run the installer and accept the defaults. On Windows, make sure the WSL2 option is selected if prompted. This is Docker's recommended backend.
3. Restart your machine if the installer asks you to.
4. Open Docker Desktop and wait until it says "Engine running" (a green status in the bottom-left corner) before continuing.
5. Confirm it from a terminal:

Open and run this command in a separate new terminal

```
PS C:\Users\aidap> docker --version
Docker version 29.8.0, build 88096ef
PS C:\Users\aidap> docker compose version
Docker Compose version v5.5.1
PS C:\Users\aidap>
```

Cloning our aiDAPTIV-LibreChat Repository

Use our aiDAPTIV-Integration-LibreChat repository. The miphi-dev branch already includes a pre-configured librechat.example.yaml with the aiDAPTIV cache settings, so you only need to add your own .env file before starting the stack. You copy librechat.example.yaml to librechat.yaml and upload librechat.yaml into the Admin Panel in step 5 below to add the aiDAPTIV endpoint to LibreChat.

GitHub - Phison-MiPhi-Collaboration/aiDAPTIV-Integration-LibreChat at miphi-dev · GitHub

1. Clone the repository and switch into it:

```
git clone -b miphi-dev https://github.com/Phison-MiPhi-Collaboration/aiDAPTIV-
Integration-LibreChat librechat
cd librechat
```

2. Create your environment file. LibreChat won't start without a .env. It holds the MongoDB connection string, session/JWT secrets, and any API keys. So copy the example that ships in the repo, then add the Admin Panel session secret to it before step 3 (see "Adding the Admin Panel session secret" below):

```
cp .env.example .env
# Windows: copy .env.example .env
```

3. Create your own librechat.yaml. LibreChat ships a template file called librechat.example.yaml. Copying it gives you a real librechat.yaml that LibreChat will actually read:

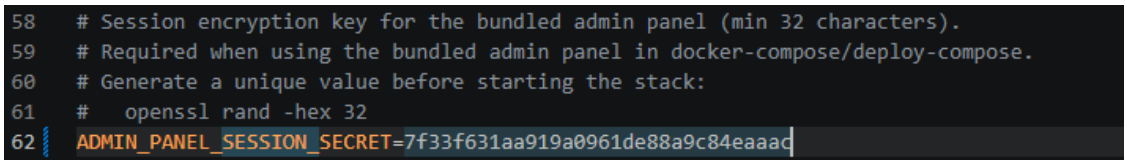
```
cp librechat.example.yaml librechat.yaml
# Windows: copy librechat.example.yaml librechat.yaml
```

Default ports

Service	Default port and address
LibreChat (chat UI)	3080 — http://localhost:3080
LibreChat Admin Panel	3000 — http://localhost:3000
aiDAPTIV llama.cpp server	8080 — http://127.0.0.1:8080

Adding the Admin Panel session secret to your .env

The Admin Panel is bundled with LibreChat and will not start without a session secret. a random string of at least 32 characters that it uses to encrypt login sessions. You set it in your .env file before starting the stack.



The Admin Panel block in .env — the setting is on line 62 in this example (your line number may differ)

1. Open your .env file in any text editor and find the comment "Session encryption key for the bundled admin panel". The ADMIN_PANEL_SESSION_SECRET line sits directly under it (line 62 in the example above).
2. Choose a value. Either generate your own (recommended) or use the default value in step 3.

To generate your own, open a terminal and run the command below (macOS, Linux, WSL, or Git Bash on Windows). It prints a 64-character random string — copy it:

```
openssl rand -hex 32
```

On Windows PowerShell, if openssl isn't available:

```
$b = New-Object byte[] 32
[Security.Cryptography.RandomNumberGenerator]::Create().GetBytes($b)
($b | ForEach-Object { $_.ToString("x2") }) -join ""
```

3. Put the value on the ADMIN_PANEL_SESSION_SECRET line — no spaces and no quotes. To use the default value instead, use exactly this line:

```
ADMIN_PANEL_SESSION_SECRET=7f33f631aa919a0961de88a9c84eaaac
```

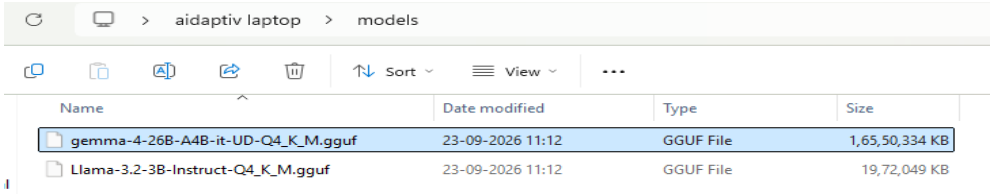
4. Save .env, then start the stack (or restart it if it was already running):

```
docker compose down
docker compose up -d
```

> **NOTE**
>
> The default value is fine for a local test on your own machine. Anyone who has this manual knows it, so generate your own before LibreChat is reachable by other users.

The rest of these prerequisites should also be in place before you start LibreChat:

6. Your LLM model file (a .gguf file) should be saved on the disk that your aiDAPTIV middleware can reach.
7. The aiDAPTIV middleware is installed as per its own setup guide, and the aiDAPTIV NVMe cache path it uses is configured within the middleware.



8. Start the LLM model server (via the aiDAPTIV middleware) pointed at that models folder, and confirm it reports a listening, OpenAI-compatible address, e.g. **http://127.0.0.1:8080/v1**. LibreChat will connect to this address in Part 3.

> **NOTE**
>
> LibreChat itself does not load the model. It talks to whatever OpenAI-compatible endpoint your aiDAPTIV Middleware exposes. Get that server running and confirmed first.

Part 2: The aiDAPTIV Cache Configuration

This section explains the two aiDAPTIV cache parameters. They live in the librechat.yaml file shown below (already present in our repository), which you upload to the Admin Panel in Part 3 — or you can type the same two parameters in by hand.

1. Inside the custom endpoint block of librechat.yaml, the addParams: section already contains the two parameters required to offload and reuse the KV cache:

librechat.yaml

```yaml
version: 1.3.5
cache: true

endpoints:
  custom:
    - name: "aiDAPTIV+"
      apiKey: "no-key"
      baseURL: "http://host.docker.internal:8080/v1"

      models:
        default:
          - "gemma-4"
        fetch: false

      titleConvo: true
      titleModel: "gemma-4"
      titleMethod: "completion"
      titleTiming: "immediate"
      modelDisplayLabel: "aiDAPTIV+"

      addParams:
        cache_prompt: true
        offload_folder_name: "benchmark"
```

What the two keys do:

Key	What it does
cache_prompt	Tells the llama-server to save the KV cache generated during prefill instead of discarding it once the reply is sent, and to check for a matching cache before recomputing on the next request.
offload_folder_name	The subfolder name, under your aiDAPTIV NVMe cache root, where this model's KV-cache files are written (in this example, "benchmark-10"). You will see this exact name again in Part 5 when you go looking for the cache files on disk.

Because these are OpenAI-compatible request parameters, addParams (shown as Additional parameters in the Admin Panel) merges them directly into the request body LibreChat sends to your aiDAPTIV llama.cpp server on every completion request.

2. If you ever need a different offload_folder_name (e.g. to keep separate caches per model or benchmark run), change it under the endpoint's Additional parameters in the Admin Panel or edit that one line in your librechat.yaml and upload it again:

> **NOTE**
>
> With our repository the librechat.yaml already contains these settings, so you can go straight from Part 1 to Part 3 and upload it — this Part is here for reference, not as a required editing step.

Part 3: Connecting LibreChat to Your LLM Model

With the model server running (Part 1), add the aiDAPTIV llama.cpp server to LibreChat as a custom endpoint from the Admin Panel. There are two ways to do it: upload your librechat.yaml (fastest) or enter the values manually. Afterwards, check the endpoint in the LibreChat chat UI.

> **NOTE**
>
> The first account you register becomes the admin account — there is no default login. This is also the account you use to sign in to the Admin Panel (http://localhost:3000).

Add the custom endpoint in the Admin Panel

The aiDAPTIV llama.cpp server is added as a custom endpoint from the Admin Panel. It is the same OpenAI-compatible API on port 8080.

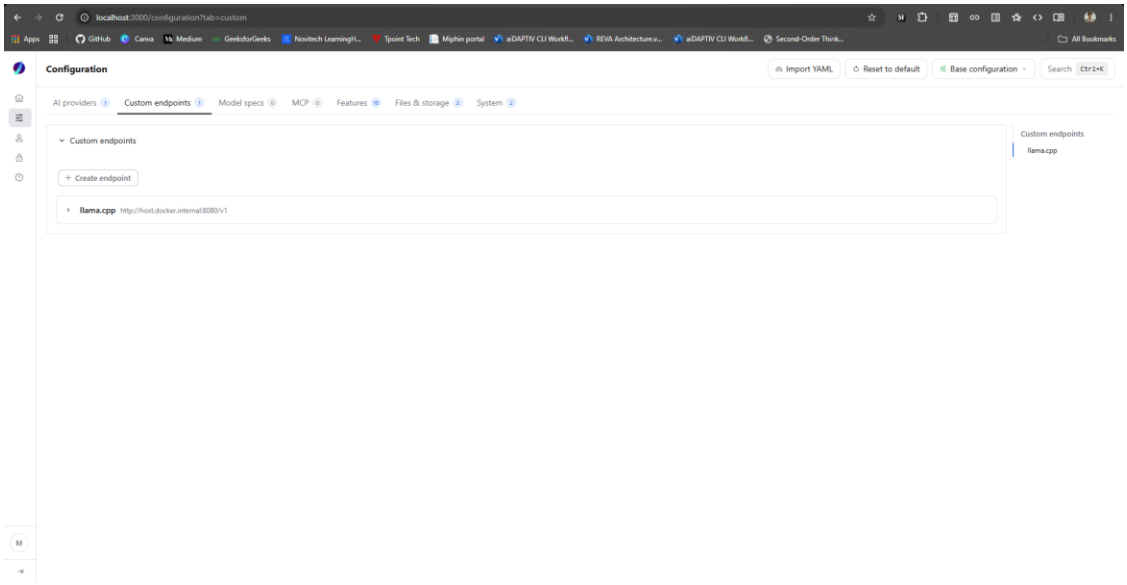
Users should first register an account in http://localhost:3080 and then access the Admin Panel at http://localhost:3000 .

Open the Admin Panel at http://localhost:3000 (a separate address from the LibreChat chat UI on port 3080), sign in with your admin account, and choose Configuration (the sliders icon in the left sidebar). Then use either Option 1 or Option 2 below.

Option 1 — Upload your librechat.yaml (fastest)

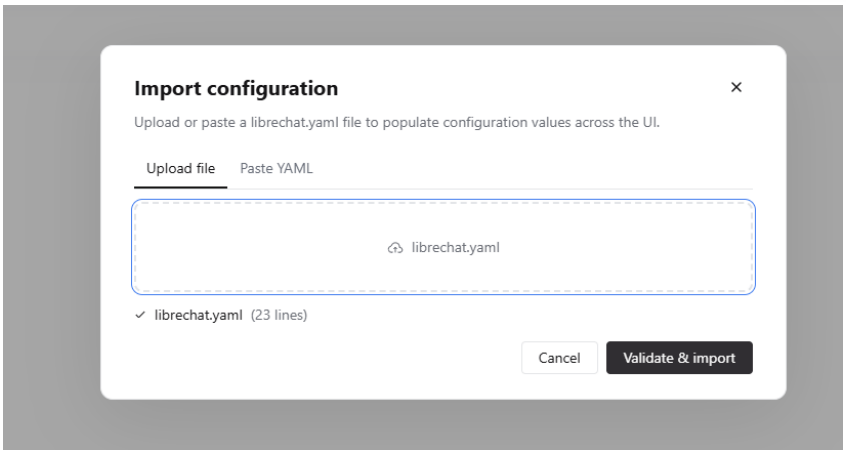
The Admin Panel can import a librechat.yaml file — for example the one from our repository (the same block shown in Part 2) — so nothing needs to be retyped.

1. Go to Configuration → Custom endpoints and click Import YAML at the top right.



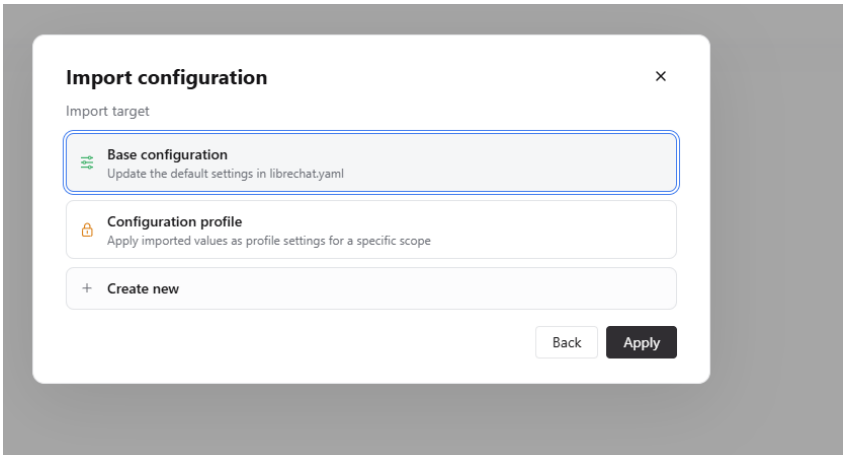
Configuration → Custom endpoints, with the Import YAML button at the top right

2. In the Import configuration window, use the Upload file tab to select your librechat.yaml (or open the Paste YAML tab and paste its contents). The file name and line count appear once it is loaded. then click Validate & import.



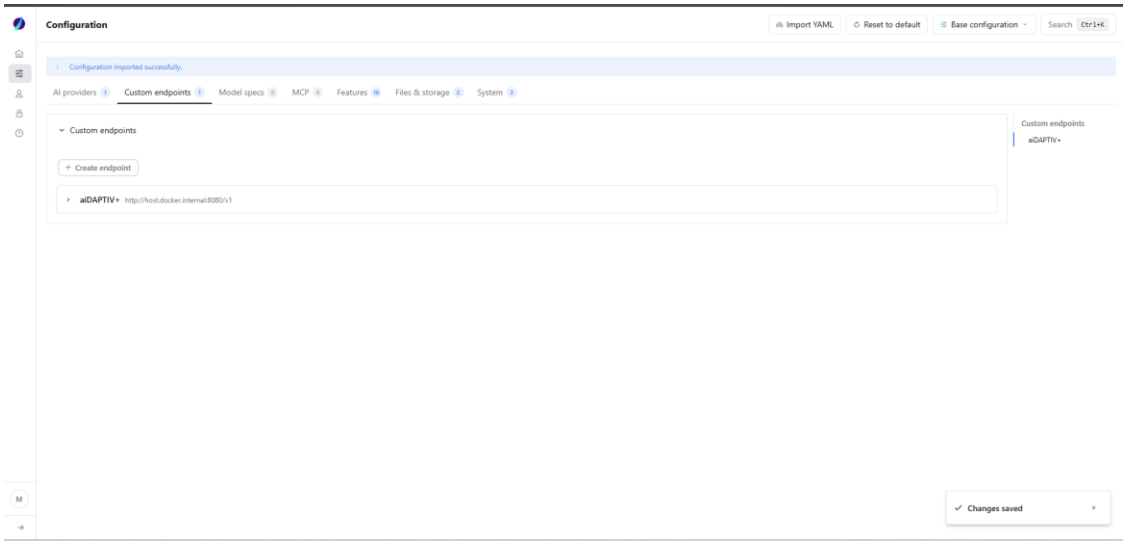
Import configuration: upload or paste librechat.yaml, then Validate & import

3. Choose the import target. Select Base configuration (updates the default settings) and click Apply.



Import target: choose Base configuration, then Apply

4. A "Configuration imported successfully" banner and a "Changes saved" message appear, and the endpoint (aiDAPTIV in this example) is listed under Custom endpoints with its base URL.



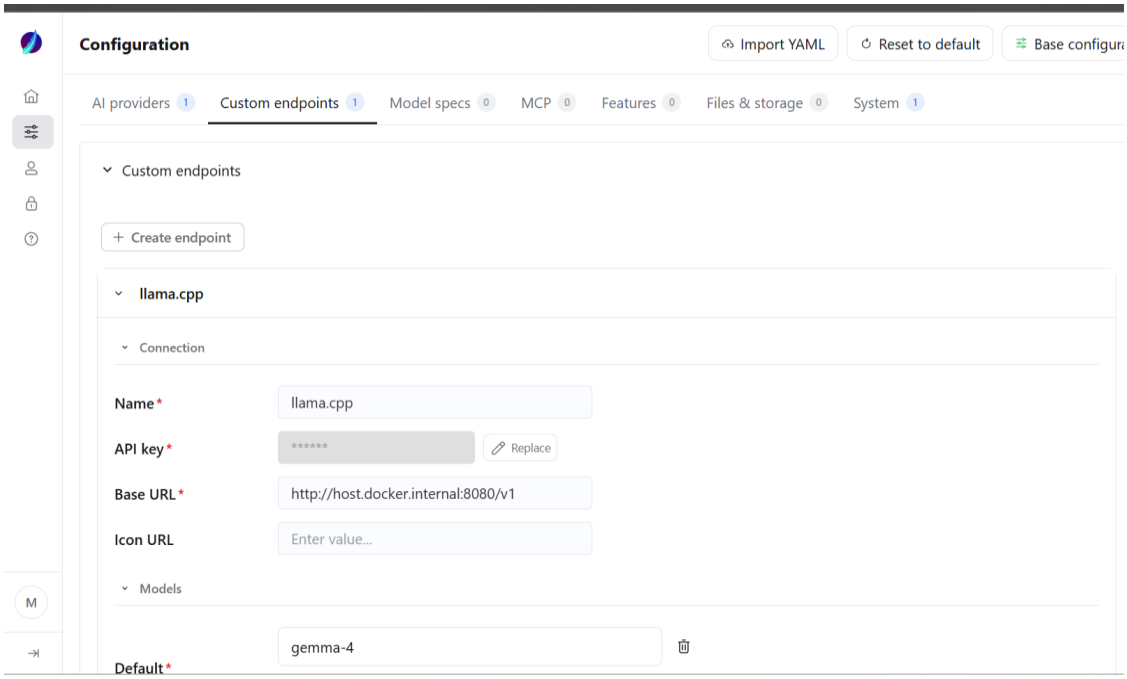
Import complete: the endpoint now appears under Custom endpoints

5. Expand the endpoint and check Advanced → Additional parameters: cache_prompt and offload_folder_name should be listed, as shown in Option 2 below.

Option 2 — Enter the values manually

Use this if you don't have a librechat.yaml to import, or you prefer to fill in the form yourself.

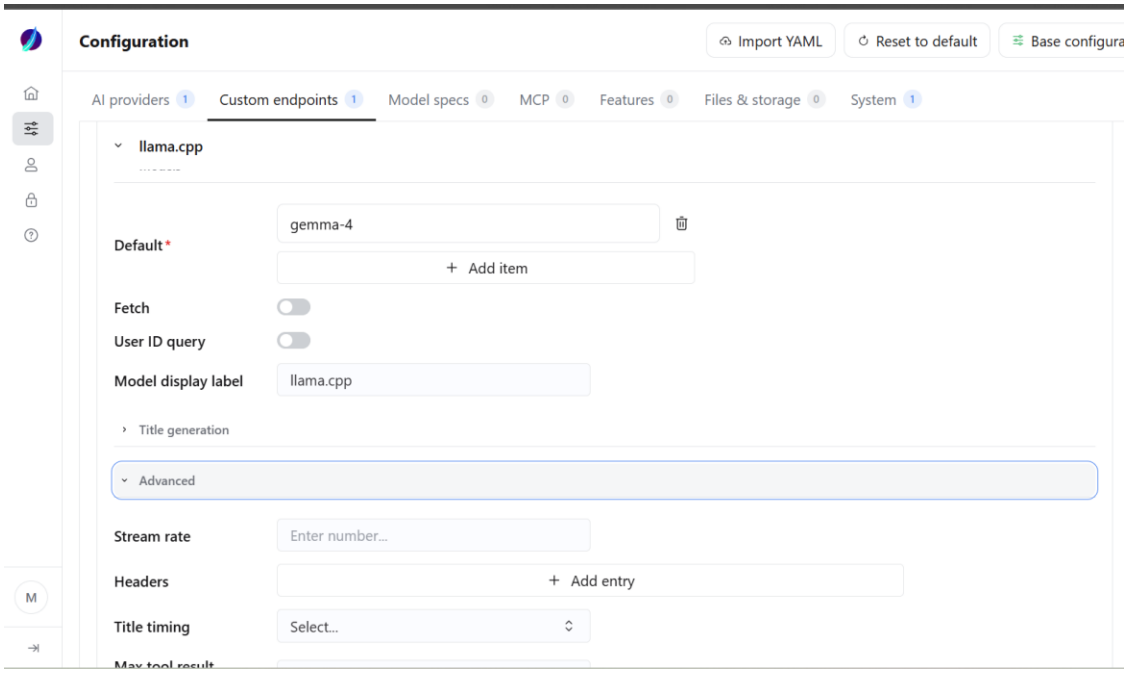
1. Under Configuration → Custom endpoints, click Create endpoint. Under Connection, fill in the fields below:



Configuration → Custom endpoints → Connection: Name, API key, Base URL and Models

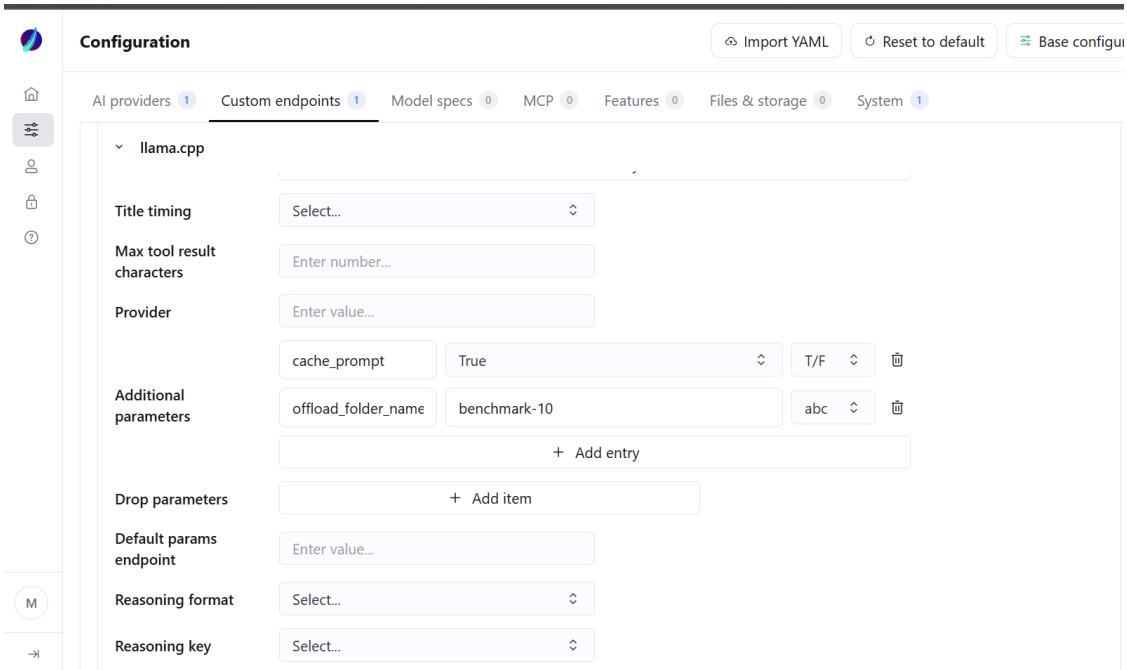
Field	What to enter
Name	A label for the endpoint — llama.cpp in this example. This is the name you will pick in the endpoint selector.
API key	A placeholder value unless your llama-server requires one. Once saved it is shown masked (******); use Replace to change it.
Base URL	http://host.docker.internal:8080/v1 — use host.docker.internal, not localhost or 127.0.0.1, because LibreChat runs in a container. Port 8080 is the aiDAPTIV llama.cpp server.
Icon URL	leave empty.

2. Under Models, click Add item and enter your model name exactly as your llama-server serves it (gemma-4 in this example). Set a Model display label if you want a friendlier name in the selector.



Models: Default model list, Fetch toggle and Model display label

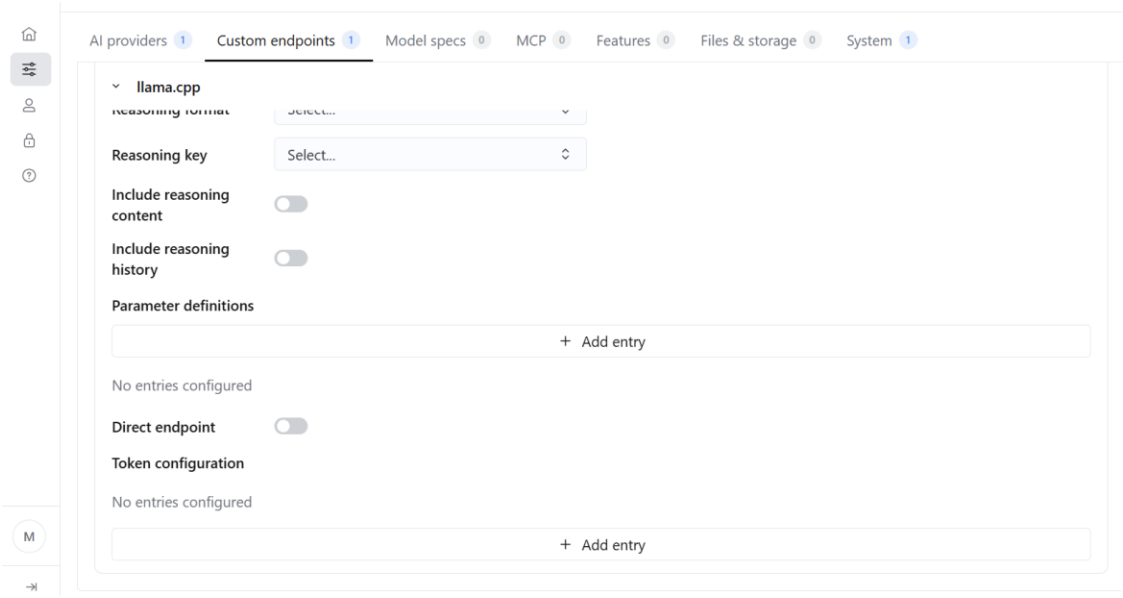
3. Expand Advanced and scroll to Additional parameters. Click Add entry twice to add the two aiDAPTIV cache parameters — these are the same two keys described in Part 2:



Advanced → Additional parameters: cache_prompt and offload_folder_name

Parameter	Value and type
cache_prompt	True - set the type selector to T/F so it is sent as a true/false value, not as text.
offload_folder_name	Your cache folder name (benchmark-10 in this example) - set the type selector to abc (text). This is the folder you will look for in Part 5.

4. Leave every other setting (Drop parameters, Reasoning, Direct endpoint, Token configuration, etc.) at its default, then save the configuration.

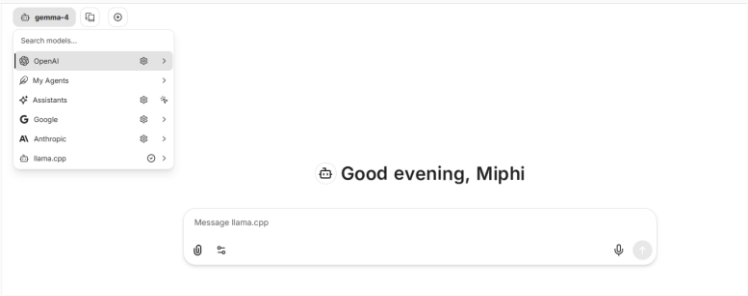


The remaining Advanced options can stay at their defaults

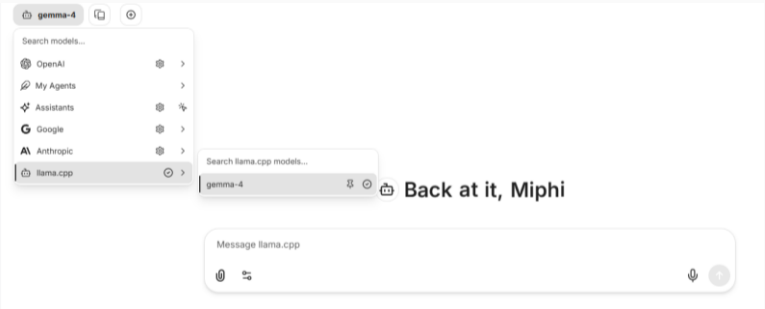
Check the endpoint in LibreChat

Once the endpoint is saved, confirm LibreChat picked it up.

1. Open LibreChat in your browser (http://localhost:3080) and, from the top of the message box, click the endpoint selector.



2. From the list of Available Endpoints, choose llama.cpp — the custom endpoint you just added.



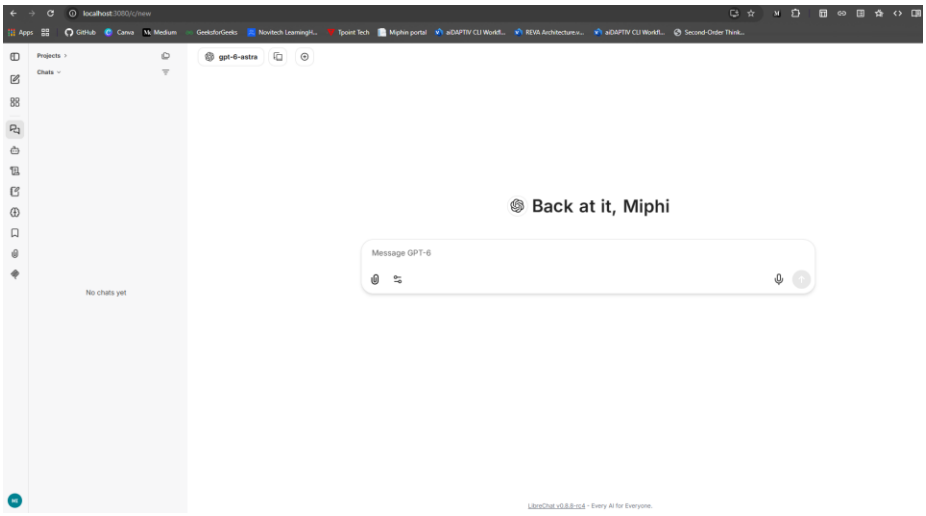
3. Confirm the fields it was saved with:

Field	What it's set to
Base URL	http://host.docker.internal:8080/v1 — use host.docker.internal, not localhost or 127.0.0.1. LibreChat's containers run on their own network and cannot see your host machine's localhost; Docker's host.docker.internal name resolves to it instead.
API Key	Leave as the placeholder value you entered or imported, unless your llama-server is configured to require one.
Models	Your model name(s), exactly as reported by your aiDAPTIV llama-server's /v1/models endpoint.

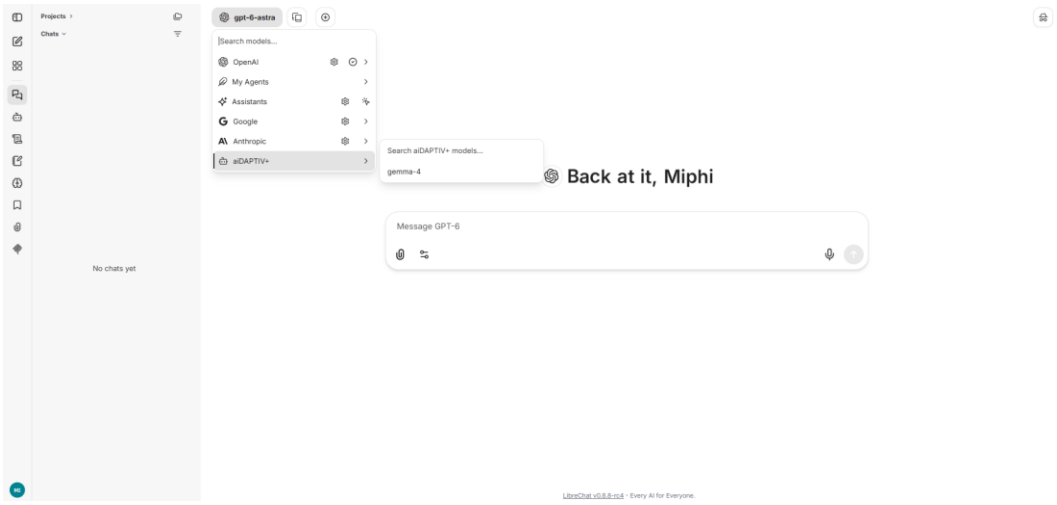
4. Select your model so it is checked and active.

Part 4: Getting Started

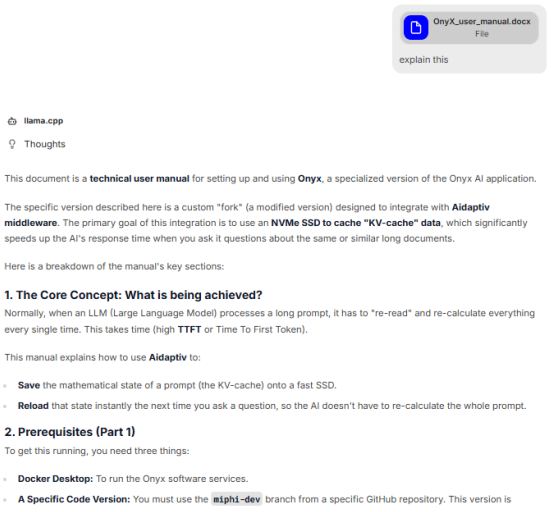
1. Open http://localhost:3080 in your browser.



2. Confirm the endpoint and model selector (top of the message box) shows the custom endpoint you added in Part 3 and the model you connected.



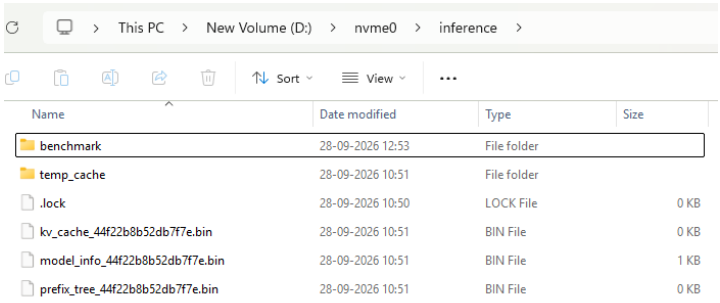
3. Type your prompt — or attach a file first, then type your question — and send it.



Part 5: Verifying the aiDAPTIV KV-Cache Offload Is Working

Check the cache folder on disk

After sending at least one prompt, browse to your aiDAPTIV NVMe cache root plus the offload_folder_name you set in Part 2 or Part 3 (for example, benchmark-10). New files confirm the KV cache is actually being written to the aiDAPTIV SSD, not just held in VRAM.



Confirm a cache hit

Ask a prompt for the first time. Its KV cache is saved to the benchmark-10 folder in aiDAPTIV. Send the same (or an overlapping) prompt again and check the llama-server log for the next request:

```
0.21.634.215 I srv  params_from_: Chat format: peg-gemma4
0.22.513.100 I slot get_availabl: id  2 | task -1 | selected slot by LRU, t_last = -1
0.22.513.181 I slot launch_slot_: id  2 | task 2 | processing task, is_child = 0
0.22.513.193 I slot update_slots: id  2 | task 2 | Hit token cnt (GPU): 0
0.22.928.411 I slot update_slots: id  2 | task 2 | Hit token cnt (DRAM): 0
0.22.928.422 I slot update_slots: id  2 | task 2 | Hit token cnt (CACHE): 12064
[MDW][Info][Runtime] Save 0 tokens to folder 'benchmark'
```

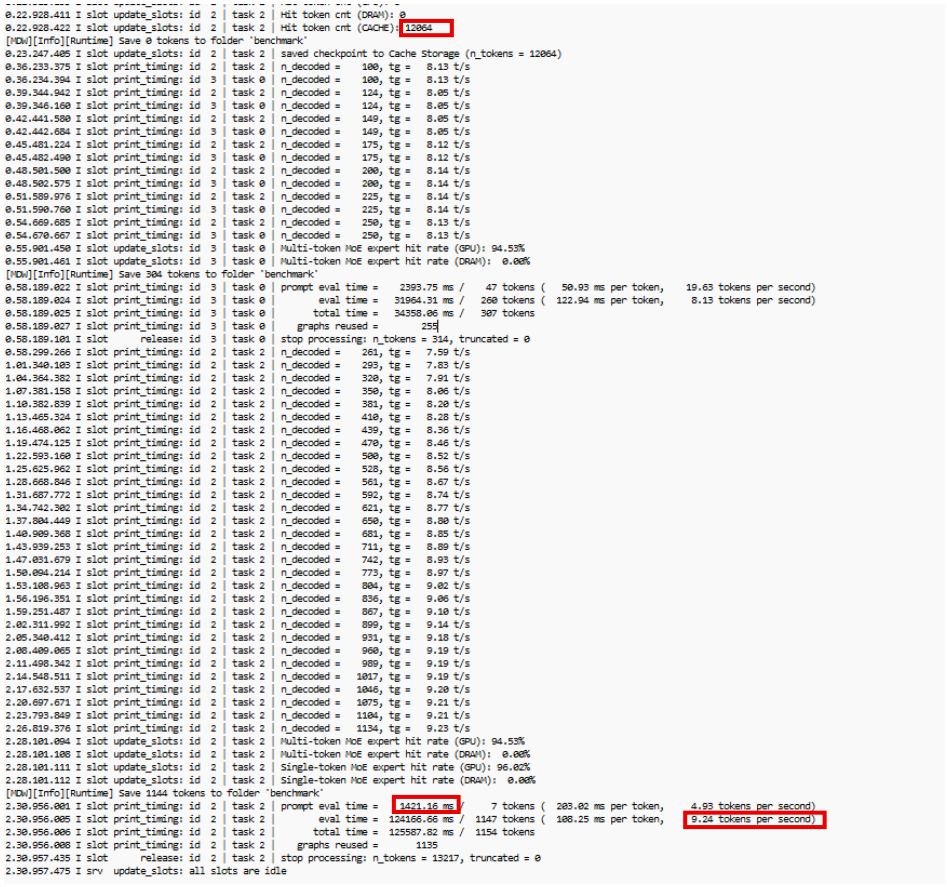
A large, non-zero Hit token cnt (CACHE) value is direct proof that those tokens were retrieved from the aiDAPTIV NVMe cache instead of being recomputed on the GPU. A closing line such as "Save 0 tokens to folder 'benchmark-10'" confirms nothing new needed to be cached because it was already there. This lower recompute workload is what produces the faster Time To First Token you'll see on repeated or overlapping prompts.

> **ℹ If your second run still shows Hit token cnt (CACHE): 0**
>
> Double-check that: cache_prompt and offload_folder_name are saved under the endpoint's Additional parameters in the Admin Panel; the prompt overlaps meaningfully with the first one; and the benchmark folder has free space on the aiDAPTIV SSD.

Part 6: Benchmark Comparison — TTFT, TPS & Cache Hit

Run 2 — Cache Hit (KV Reuse)

From our benchmarking test. A previous run had already built up a token cache for this prompt; this run reused it:



TTFT: 1,421.16 ms • TPS: 9.24 tokens/sec • Hit Cache (CACHE): 12,064 tokens

Metric	Run 2 — Warm (cache hit)
Hit token cnt (CACHE)	12,064 tokens
TTFT (prompt eval time)	1,421.16 ms
TPS (decode / eval speed)	9.24 tokens/sec